

STUDENT PROJECT GUIDE & DOCUMENTATION

Building and Deploying an Ohm's Law Calculator with Python & Tkinter

Table of Contents

TABLE OF CONTENTS	1
1. PROJECT OVERVIEW & GRADING RUBRIC	2
1.1 INTRODUCTION TO THE PROJECT	2
1.2 GRADING BREAKDOWN MATRIX	2
2. REQUIREMENTS & SOFTWARE INSTALLATION	2
3. STEP-BY-STEP CODE INSTRUCTIONS	2
3.1 STEP 1: IMPORTING REQUIRED LIBRARIES	2
3.2 STEP 2: CREATING THE MAIN APPLICATION WINDOW	3
3.3 STEP 3: SETTING UP TRACKING VARIABLES	3
3.4 STEP 4: DESIGNING THE MODE SELECTION BUTTONS	3
3.5 STEP 5: CREATING THE INPUT BOXES	3
3.6 STEP 6: ADDING ACTION AND CLEAR BUTTONS	4
3.7 STEP 7: MAKING THE WINDOW CHANGE AUTOMATICALLY	4
3.8 STEP 8: WRITING THE MATH AND ERROR PROTECTION LOGIC	5
4. COMPLETE OPERATIONAL SOURCE CODE	6
5. TURNING CODE INTO A STANDALONE WINDOWS APP (.EXE)	6
5.1 WHAT IS AN EXECUTABLE APP?	6
5.2 CHECKING YOUR SYSTEM ENVIRONMENT	6
5.3 INSTALLING PYINSTALLER VIA TERMINAL	6
5.4 CREATING THE STANDALONE APP (.EXE)	6
5.5 FINDING AND TESTING YOUR NEW APP	7
6. SHARING YOUR APP SAFELY	7
7. TROUBLESHOOTING COMMON PROBLEMS	7

1. Project Overview & Grading Rubric

1.1 Introduction to the Project

In this project, you will design and build a desktop app called the Ohm's Law Calculator. This app allows users to calculate Voltage (V), Current (I), or Resistance (R) using Ohm's Law formulas. Your app will include built-in safety features to prevent common mistakes. For example, it will stop the app from crashing if a user types letters instead of numbers, inputs numbers below zero, or accidentally creates a short-circuit by dividing by zero.

1.2 Grading Breakdown Matrix

Your project will be graded out of 30 marks based on the following evaluation layout constraints:

Grading Component	Weight	What You Need to Do
Window Layout & Design	8 Marks	Create the main window properly, group sections neatly, and make sure text items and boxes do not overlap.
Automatic Updates	6 Marks	Make the entry boxes lock or unlock automatically whenever the user changes what they want to calculate.
Math & Formulas	6 Marks	Use correct formulas for Ohm's Law ($V = I * R$, $I = V / R$, $R = V / I$) and display clean decimal answers.
Error Protection	6 Marks	Use code safety blocks (try-except) to catch blank entries, typed letters, negative numbers, and dividing by zero.
Clean Formatting	4 Marks	Use clear names for your variables, add simple notes (comments) to explain your code, and organize your files code clean.

2. Requirements & Software Installation

Before you start writing code, make sure your computer has the right software installed. Everything we use for the layout comes built-in with Python, so you do not need to download extra coding tools for the app itself.

- Python Environment: Install a modern version of Python (Version 3.10 or newer is best).
- Tkinter Framework: This is the tool package used to make the visual windows. It comes free with standard Python installations.
- Coding Editor: You can write your code using standard Python IDLE, Visual Studio Code, or PyCharm.

3. Step-by-Step Code Instructions

3.1 Step 1: Importing Required Libraries

Your app needs three basic tools from Python. 'tkinter' creates the visual shapes, 'tk' gives the buttons and boxes a clean design, and 'messagebox' creates pop-up warning windows.

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
```

3.2 Step 2: Creating the Main Application Window

Every window app needs a main base frame. Here, we build the background window, give it a title, set a fixed window size, and stop the user from stretching it out of shape.

```
class OhmsLawCalculator:
    def __init__(self, root):
        self.root = root
        self.root.title("Ohm's Law (Voltage) Calculator")
        self.root.geometry("420x340")
        self.root.resizable(False, False) # Stops user resizing
```

3.3 Step 3: Setting Up Tracking Variables

To help the app remember exactly what value the student wants to calculate, we use a special tracking variable called a 'StringVar'. It holds a value of 'V', 'I', or 'R' and alerts the app instantly when a new option is clicked.

```
# Tracks the selected calculation mode: "V" (Voltage), "I" (Current), "R"
(Resistance)
self.solve_for = tk.StringVar(value="V")
```

3.4 Step 4: Designing the Mode Selection Buttons

We create a neat frame box with a text border. Inside it, we put three choice buttons (Radiobuttons) side-by-side. When clicked, they update our tracking variable and change which entry fields are active.

```
# Section 1: Mode Selection Box
mode_frame = ttk.LabelFrame(self.root, text=" 1. Select What to Calculate ",
padding=10)
mode_frame.pack(fill="x", padx=15, pady=10)

ttk.Radiobutton(mode_frame, text="Voltage (V)", variable=self.solve_for,
                value="V", command=self.update_fields).pack(side="left",
expand=True)
ttk.Radiobutton(mode_frame, text="Current (I)", variable=self.solve_for,
                value="I", command=self.update_fields).pack(side="left",
expand=True)
ttk.Radiobutton(mode_frame, text="Resistance (R)", variable=self.solve_for,
                value="R", command=self.update_fields).pack(side="left",
expand=True)
```

3.5 Step 5: Creating the Input Boxes

Next, we arrange text labels, typing boxes, and electrical units into rows and columns like a grid layout. We use padding spaces to keep the layout highly readable.

```
# Section 2: Input Box Frame
self.input_frame = ttk.LabelFrame(self.root, text=" 2. Enter Known Values ",
padding=10)
self.input_frame.pack(fill="x", padx=15, pady=5)

# Row 0: Voltage Widgets
self.lbl_v = ttk.Label(self.input_frame, text="Voltage (V):")
self.lbl_v.grid(row=0, column=0, sticky="w", pady=5)
self.ent_v = ttk.Entry(self.input_frame, font=("Helvetica", 11))
```

```

self.ent_v.grid(row=0, column=1, fill="x", expand=True, padx=10, pady=5)
self.unit_v = ttk.Label(self.input_frame, text="Volts (V)")
self.unit_v.grid(row=0, column=2, sticky="w")

# Row 1: Current Widgets
self.lbl_i = ttk.Label(self.input_frame, text="Current (I):")
self.lbl_i.grid(row=1, column=0, sticky="w", pady=5)
self.ent_i = ttk.Entry(self.input_frame, font=("Helvetica", 11))
self.ent_i.grid(row=1, column=1, fill="x", expand=True, padx=10, pady=5)
self.unit_i = ttk.Label(self.input_frame, text="Amperes (A)")
self.unit_i.grid(row=1, column=2, sticky="w")

# Row 2: Resistance Widgets
self.lbl_r = ttk.Label(self.input_frame, text="Resistance (R):")
self.lbl_r.grid(row=2, column=0, sticky="w", pady=5)
self.ent_r = ttk.Entry(self.input_frame, font=("Helvetica", 11))
self.ent_r.grid(row=2, column=1, fill="x", expand=True, padx=10, pady=5)
self.unit_r = ttk.Label(self.input_frame, text="Ohms ( $\Omega$ )")
self.unit_r.grid(row=2, column=2, sticky="w")

```

3.6 Step 6: Adding Action and Clear Buttons

We add two standard control buttons. The 'Calculate' button processes the numbers, while the 'Clear' button wipes away text so students can quickly start a fresh math operation.

```

# Section 3: Action Buttons
btn_frame = ttk.Frame(self.root, padding=10)
btn_frame.pack(fill="x", padx=15)

self.btn_calc = ttk.Button(btn_frame, text="Calculate", command=self.calculate)
self.btn_calc.pack(side="left", expand=True, fill="x", padx=5)

self.btn_clear = ttk.Button(btn_frame, text="Clear", command=self.clear_fields)
self.btn_clear.pack(side="right", expand=True, fill="x", padx=5)

# Output Results Banner
self.lbl_result = ttk.Label(self.root, text="Result will appear here.",
                             font=("Helvetica", 11, "bold"), foreground="#002060",
                             anchor="center")
self.lbl_result.pack(fill="x", pady=10)

```

3.7 Step 7: Making the Window Change Automatically

This critical block locks or unlocks the text cells based on your option choice. When solving for Voltage, it clears the Voltage field and disables it so you can only type into Current and Resistance.

```

def update_fields(self):
    mode = self.solve_for.get()
    self.ent_v.config(state="normal")
    self.ent_i.config(state="normal")
    self.ent_r.config(state="normal")

    if mode == "V":
        self.ent_v.delete(0, tk.END)
        self.ent_v.config(state="disabled")
        self.lbl_result.config(text="Enter Current & Resistance, then click

```

```

Calculate.")
    elif mode == "I":
        self.ent_i.delete(0, tk.END)
        self.ent_i.config(state="disabled")
        self.lbl_result.config(text="Enter Voltage & Resistance, then click
Calculate.")
    elif mode == "R":
        self.ent_r.delete(0, tk.END)
        self.ent_r.config(state="disabled")
        self.lbl_result.config(text="Enter Voltage & Current, then click Calculate.")

def clear_fields(self):
    self.ent_v.config(state="normal")
    self.ent_i.config(state="normal")
    self.ent_r.config(state="normal")
    self.ent_v.delete(0, tk.END)
    self.ent_i.delete(0, tk.END)
    self.ent_r.delete(0, tk.END)
    self.update_fields()

```

3.8 Step 8: Writing the Math and Error Protection Logic

This is the core engine. It reads the boxes, runs calculations, and uses try-except blocks to catch empty items, letters, numbers below zero, or divisions by zero gracefully.

```

def calculate(self):
    mode = self.solve_for.get()
    try:
        if mode == "V":
            i = float(self.ent_i.get())
            r = float(self.ent_r.get())
            if i < 0 or r < 0: raise ValueError("Negative Value")
            self.lbl_result.config(text=f"Calculated Voltage (V) = {i * r:.4f} V")
        elif mode == "I":
            v = float(self.ent_v.get())
            r = float(self.ent_r.get())
            if v < 0 or r < 0: raise ValueError("Negative Value")
            if r == 0: raise ZeroDivisionError("Short circuit: Resistance cannot be 0
Ω.")
            self.lbl_result.config(text=f"Calculated Current (I) = {v / r:.4f} A")
        elif mode == "R":
            v = float(self.ent_v.get())
            i = float(self.ent_i.get())
            if v < 0 or i < 0: raise ValueError("Negative Value")
            if i == 0: raise ZeroDivisionError("Open circuit: Current cannot be 0 A.")
            self.lbl_result.config(text=f"Calculated Resistance (R) = {v / i:.4f} Ω")
    except ValueError as err:
        if "Negative" in str(err):
            messagebox.showerror("Validation Error", "Electrical dimensions cannot be
negative.")
        else:
            messagebox.showerror("Input Error", "Please enter valid numerical digits in
all fields.")
    except ZeroDivisionError as err:
        messagebox.showerror("Mathematical Error", str(err))

```

5. Turning Code into a Standalone Windows App (.exe)

5.1 What is an Executable App?

When you finish writing your code, it exists as a standard text script ending in '.py'. To make this script run, any computer trying to open it must already have Python pre-installed.

To share projects cleanly with family, friends, or teachers who do not have Python installed, we must perform an operation called Turning Code into a Standalone App (Executable Binary Packaging). We use a popular tool named PyInstaller. It automatically scans your script, collects all required system pieces, builds in a hidden tiny copy of the Python engine, and bundles it all into a single click-and-run file ending in '.exe'.

Feature / Dimension	Source Python Script (.py)	Standalone App Icon (.exe)
Target Dependencies	Requires full Python installation & libraries setup on target computer.	Completely standalone. Runs instantly without installing Python.
Code Exposure	Raw readable text format. Anyone can view or change your code logic directly.	Hidden machine code layer. Keeps your logic formulas private from view.
Sharing Method	Must copy individual files and teach users how to use terminal commands.	Packaged into a single, straightforward click-and-run app icon.

5.2 Checking Your System Environment

Before downloading any expansion tools, make sure your computer command program recognizes Python's package manager tool (known as 'pip').

Instruction: Open your Windows Command Prompt by opening your Start menu, typing 'cmd', and hitting the Enter key. In the black window that appears, type the command below exactly and press Enter:

```
pip --version
```

Expected Outcome: If working properly, the text window will echo back version stats tracking where pip is located. If you see an error reading 'command not found' or 'not recognized', your Python paths are broken. Ask your lab instructor to check your environmental variable configurations.

5.3 Installing PyInstaller via Terminal

With your terminal connections checked, download the compiler package safely from the official global library using your internet connection.

Instruction: Inside that same command window, type this command and press Enter:

```
pip install pyinstaller
```

Watch the setup text lines advance down your row space until you see a clear line reading:

```
Successfully installed pyinstaller-X.X.X
```

5.4 Creating the Standalone App (.exe)

The command terminal operates out of a specific folder pathway. You must deliberately change its focus to the exact workspace folder holding your calculator python script file.

Assume your script is named 'ohms_law_calculator.py' and is saved inside a folder pathway at:
C:\Projects\OhmCalculator

Instruction: Run the change directory command ('cd') inside your terminal panel to switch paths directly:

```
cd C:\Projects\OhmCalculator
```

Now, trigger the PyInstaller app builder using two custom flags to guarantee a clean student project setup:

- **--onefile:** Creates a single, neat standalone app icon file instead of outputting a messy folder structure full of supporting script parts.
- **--noconsole:** Suppresses the black background command shell when the app opens. Since our app has its own custom Tkinter visual window, hiding this raw panel gives it a professional look.

Instruction: Input the complete builder command exactly as written below and hit Enter:

```
pyinstaller --onefile --noconsole ohms_law_calculator.py
```

Warning: Your computer will process lines of code actively for about 30 to 90 seconds. Do not close down the window while these background processes are running.

5.5 Finding and Testing Your New App

Once the script finishes compiling and prints a final line reading 'Building EXE completed successfully', check your base directory folder project layout space. PyInstaller will have automatically generated new sub-folders inside your folder:

- **build/:** Contains internal compilation processing cache maps. You can safely ignore or delete this after compilation completes.
- **dist/:** Short for 'Distribution'. This is the target folder containing your final completed executable app artifact.

Instruction: Open your native Windows File Explorer, navigate into your project folder, and open the new 'dist/' folder. Inside, you will see your standalone app file named 'ohms_law_calculator.exe'. Double-click this icon. Your Tkinter calculator window will launch immediately without any background console screens.

6. Sharing Your App Safely

To share this completed standalone application smoothly with classmates or instructors for grading verification, follow these distribution steps:

1. **Isolate the Executable File:** You do not need to share anything inside the build/ folder, your raw script sheet, or any log files. The single executable file inside the dist/ folder is everything your user needs.
2. **Zip Container Protection:** Because raw executable files can sometimes trigger web safety blocks or email filters, it is best practice to right-click your executable file, select 'Compress to ZIP file', and share the zipped archive folder instead.
3. **Distribution Channels:** You can easily pass your compressed app folder archive via flash drives, upload it onto shared school dropboxes, or upload it to institutional cloud storage links (like OneDrive or Google Drive).

7. Troubleshooting Common Problems

Review this list of common pitfalls if you hit errors during testing or deployment:

- **App Crashes Immediately on Startup:** If the window flashes open and instantly closes, re-compile the script without using the '--noconsole' flag. This forces the black command terminal to stay open behind the app, allowing you to read any code syntax or crash errors.
- **Windows Defender SmartScreen Warnings:** Because our compiled file does not have an expensive corporate security digital signature, Windows SmartScreen will warn users with a pop-up saying 'Windows protected your

PC' the first time it opens on a new machine. Instruct users to click 'More Info' and select 'Run Anyway'.

- Architecture Mismatch Faults: PyInstaller compiles files matching the exact processor layout of the computer it was built on. An application compiled on a standard 64-bit Windows PC will run beautifully across modern endpoints, but will fail if loaded onto an ancient 32-bit lab station. Always verify target systems match.